

From plausible architecture to a defensible decision

A portable research and publication loop

Worked example · Version 1.0 · 6 October 2026

This example uses first-person authorial voice to demonstrate the requested writing style. It does not attribute personal experience, experiments or organizational outcomes to the user.

My recommendation

I propose a portable architecture loop in which the recommendation, paper, slide deck and diagrams share the same evidence and decisions. The loop combines a defined research process with local files, structural checks and substantive review. I would pilot it for substantial, recurring architecture work before adopting it broadly. Whether it improves publication quality enough to justify its extra effort remains an empirical question. [CL7] [CL11]

The decision I want this paper to enable is specific: should an author use this file-based loop for a recurring architecture problem, continue with a simpler checklist, or invest in a managed workflow? My recommendation is conditional. Use the loop when evidence, alternatives and several publication artifacts need to stay aligned across revisions or host environments. Prefer a simpler method when the task is short and its reasoning is easy to inspect directly.

I define success as a reader being able to explain the problem, the chosen mechanism, its strongest alternative, its limitations and the observation that would change the recommendation. An attractive paper or deck is useful only if it supports that decision. I therefore keep evidence and uncertainty visible through drafting and presentation design.

My central hypothesis is that durable shared records can reduce material drift between outputs. I do not claim a measured improvement, a universally accepted agent architecture or a guarantee of correct research. The supplied prototype implements the local coordination mechanics; evaluation against real authoring work is still needed. [CL6] [CL7]

The problem I am trying to solve

An architecture paper explains how a software system should work and why its design fits the problem. A slide deck presents the same decision to an audience with less reading time. Both normally depend on research, assumptions, tradeoffs, diagrams and a proposed implementation path. The author is accountable for the recommendation; engineering leaders decide whether to invest; engineers and reviewers assess feasibility and evidence.

For this design, an agent means a model-driven worker using the tools provided by its host environment. Claude Code and GitHub Copilot are possible hosts. The loop does not supply a foundation model or an internet search service. It coordinates the work an available host performs. A host without research, filesystem or rendering capabilities can complete only the corresponding parts of the workflow. [CL9]

The failure scenario I use throughout this paper is a source correction. Suppose a draft recommends a mechanism partly because a vendor appears to support a necessary capability. A later reading shows that the capability applies only under narrower conditions. The paper must change, the decision may change, and any slide relying on the old claim must change. This scenario illustrates the design problem; I am not presenting it as a measured incident or prevalence estimate.

I derive five invariants from that problem: I must not invent evidence; every pivotal choice must face a credible alternative; all four architecture views must describe the same system; both publications must preserve the same conditions; and changed canonical content must receive a new review. A canonical record is the authoritative working record from which other artifacts are derived. These invariants lead to evidence, decision, architecture and story records before they lead to a product selection. I keep structural checks and semantic review distinct. [CL3] Using shared records to preserve these conditions is the proposal I would test. [CL7]

I make the constraints explicit. This release assumes one writer per local run, document-sized inputs, a host that can work with files, and an author willing to maintain structured records. It does not provide concurrent editing, tamper-proof approval, automatic publication or independent reviewer identity. Those requirements would change the architecture.

What the evidence establishes

I use the original C4 documentation to define four views: system context, containers, components and code. A container is an application or data store; it does not necessarily mean a Docker container. A component view expands one selected container, and a code view expands one selected component. The model does not require every level for every project. I include all four because they are part of the requested deliverable. [CL1] [S1] [S2] [S3] [S4] [S5]

The workflow also draws on the evaluator-optimizer pattern: generated work is evaluated and revised. That pattern is a useful design precedent, not evidence that this particular kit improves decisions. I keep the implementation deliberately small enough to inspect and evaluate against a simpler baseline. [CL2] [S7]

Research evaluation requires different kinds of checks. Structural rules can establish that a reference exists; they cannot establish that its source supports the adjacent sentence. Substantive review must assess support, coverage, fairness and interpretation. I therefore distinguish machine checks from semantic review, which is review of meaning and evidence.

A positive structural result is never presented as proof of factual truth. [CL3] [S8]

For host integration, I verified the documented instruction-file conventions for CLAUDE.md and .github/copilot-instructions.md. The supplied adapters use those conventions as conveniences. I have not established that every host version, policy configuration or tool combination will execute this workflow unchanged. The generic master prompt remains the fallback, and real host acceptance runs remain part of the validation plan. The core has no model SDK dependency; host capabilities remain separate dependencies. [CL9] [CL4] [CL5] [S9] [S10]

The choice I am making

I compare four options against factual traceability, portability, decision usefulness, operating complexity, honest completion and required review independence. I avoid numerical option scores because the evidence does not justify precise weights.

- One prompt and manual editing: the least setup for a short note. Its limitations grow when a long research history and several artifacts must remain aligned.
- A human checklist with ordinary documents: a strong baseline. It may deliver enough rigor with less maintenance when work is occasional or one reviewer can inspect the whole decision easily.
- The proposed file-based loop: keeps evidence, choices, architecture and story in durable records; checks structural relationships repeatedly; and supports resumption across hosts. It adds files and process overhead.
- A managed workflow and approval service: may be appropriate when multiple writers, independently enforced approval or centralized integration are actual requirements. It also creates a service to build, operate and migrate.

I choose the file-based loop only for the bounded use case stated here. Decision D1 would reverse if a pilot found no improvement in material defects and more total work than the checklist. A material defect is one that could change the recommendation or prevent a reader from understanding its mechanism, scope or limitations. This definition should be agreed before the pilot. [CL7] [CL11]

Decision D2 separates structural gates from substantive review. Its strongest objection is that the same model can write the content and give it a flattering review. A fresh reviewer context may help inspect it from another perspective, but it is not independent empirical evidence. If independently enforced approval is required, I would move that authority out of the writable run folder and adopt the relevant parts of the managed-service option. [CL3] [CL8]

Decision D3 uses plain files and commands for portability. Its reversal trigger is a demonstrated inability to resume the same run or produce the required deliverables in the chosen hosts. Documented conventions do not eliminate that integration risk. I would test

the specific host configurations before relying on them operationally. [CL4] [CL5] [CL9]

My disconfirmation record includes three different kinds of work. Code inspection already shows that the local writer can modify state and checks, which limits the assurance claim. Local regression tests can probe mechanical failures such as stale reviews. Publication effectiveness and real host portability require future empirical trials. I do not relabel those planned trials as completed experiments.

How the architecture works

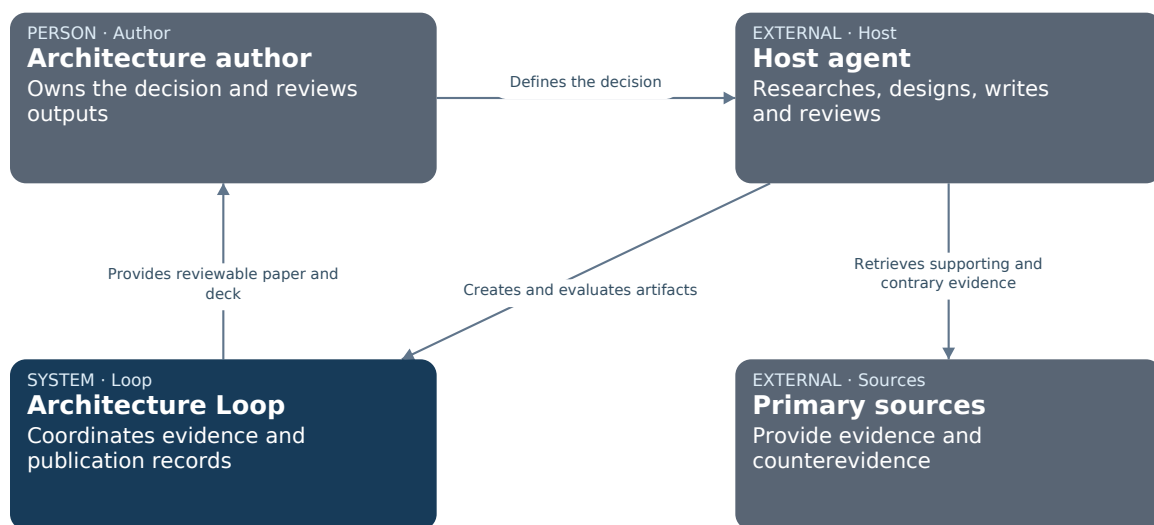
I use one registry for system elements, names, responsibilities, parent scopes and directed relationships. The views below are derived from that registry. C4 is notation independent; I use typed boxes, explicit boundaries and directed labels, with Mermaid sources retained for editing. The figures describe the supplied prototype and its host boundary. [CL1] [S6]

C1: the system in its context

The host agent retrieves evidence, performs the research and writes the artifacts. Architecture Loop supplies coordination and records. The author reviews and uses the outputs. Keeping the host outside the system boundary makes clear that the kit itself does not autonomously call a model or browse the web. [CL9]

C1 · System context

Scope: Architecture Loop · Loop



Person = accountable author. Dark blue = kit. Gray = external systems. Arrows describe directed interactions.

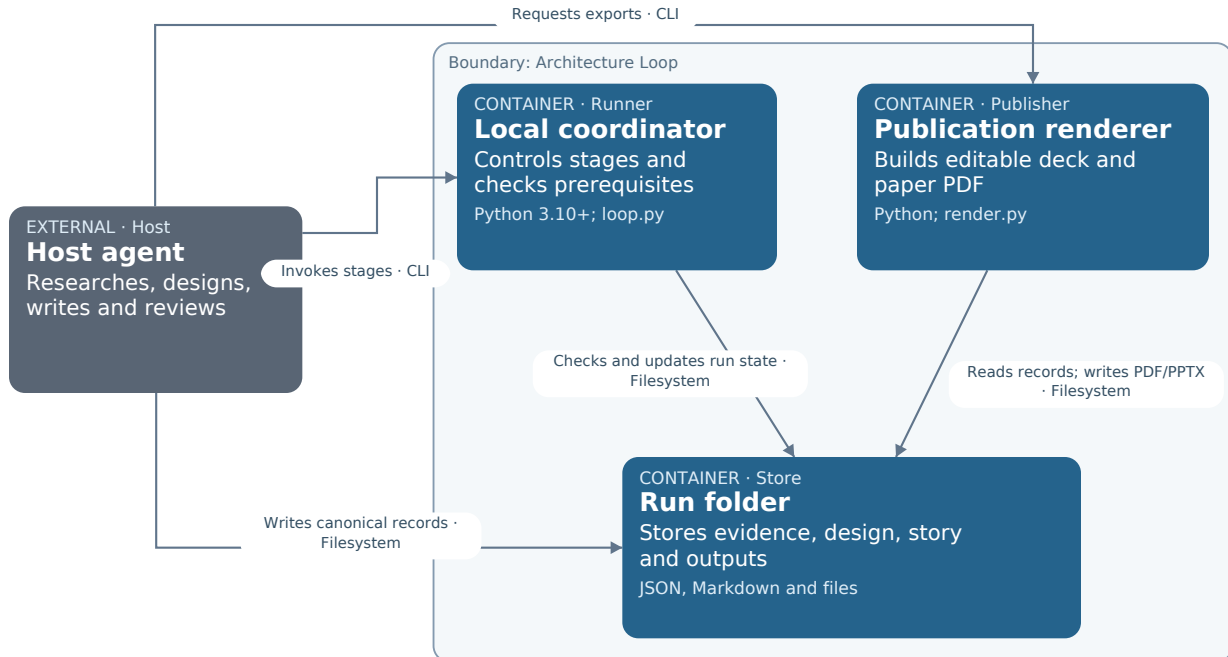
C1: the host performs intellectual work and retrieves evidence. Architecture Loop retains reviewable records and outputs. The author remains accountable for using the recommendation.

C2: the applications and data store

Inside the system I separate a local coordinator, a run folder and an optional publication renderer. The coordinator advances stages and checks prerequisites. The folder stores structured JavaScript Object Notation (JSON) records, Markdown (plain-text document source), diagrams, reviews and outputs. The renderer produces a paper PDF, an editable PowerPoint and a preview. The host invokes both programs through a command-line interface (CLI) and writes the substantive content. [CL6]

C2 · Containers within Architecture Loop

Scope: Architecture Loop · Loop



Outlined boundary = kit ownership. Blue = application or data-store container. Gray = external host. Arrow labels include protocol.

C2: the coordinator and optional renderer are separate Python applications. A local folder is the shared data store. The host invokes commands and writes content; no background model service is required.

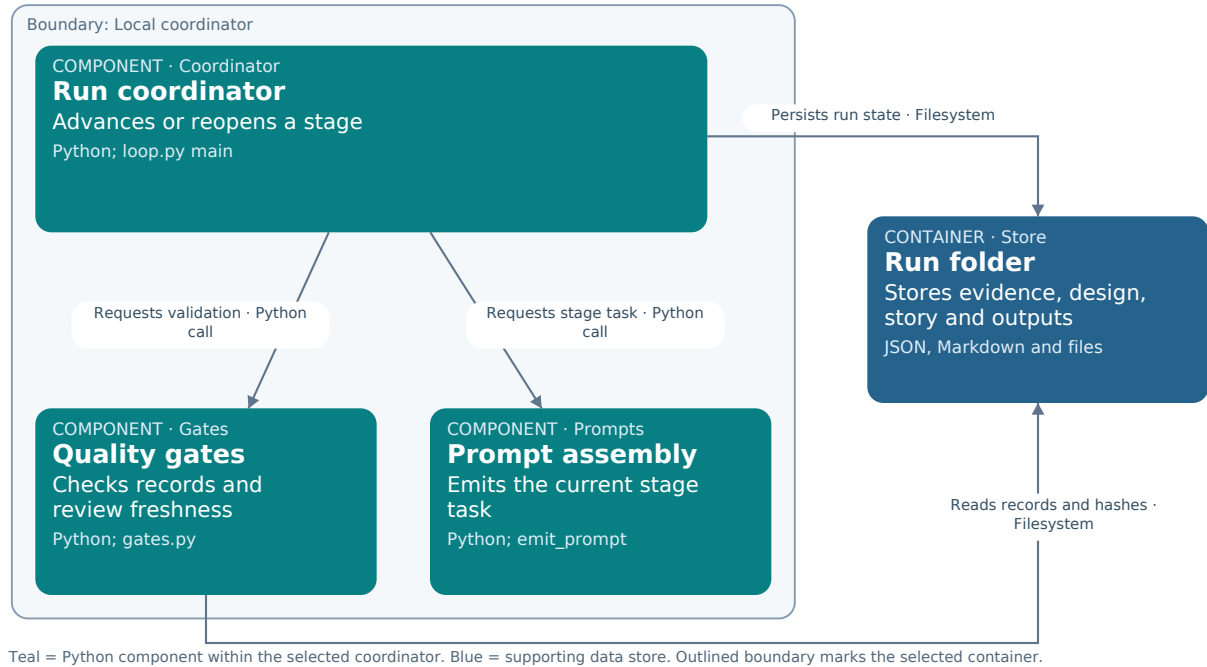
The folder is intentionally ordinary storage. It makes transfer and inspection simple, but it is not a database with multi-file transactions. The writer must finish a coherent edit before advancing. The coordinator atomically replaces its own state file; that protects one write, not an entire batch of document edits. A managed collaborative implementation would need locking or version-checked transactions.

C3: the parts of the coordinator

I expand the coordinator because stage transitions and review checks are where false completion would be introduced. The run coordinator chooses transitions; quality gates inspect the artifacts; prompt assembly emits the next stage task. The run folder remains a supporting container. The host, not the coordinator, carries out the emitted task.

C3 · Components inside the local coordinator

Scope: Local coordinator · Runner



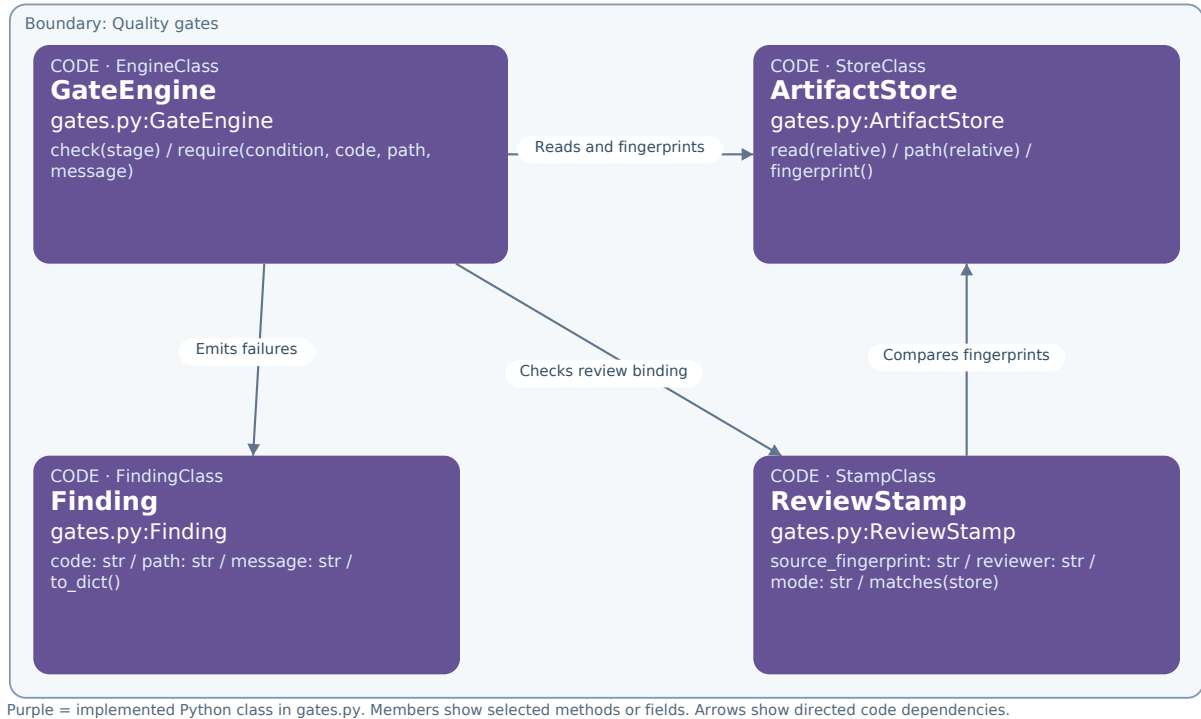
C3: the coordinator controls transitions, quality gates inspect the records, and prompt assembly supplies the next task. The folder supports all three; it is a data-store container, not a component.

C4: the implementation of quality gates

I expand the quality-gates component into four classes that exist in gates.py. GateEngine applies cumulative validation and returns Finding records. ArtifactStore reads scoped paths and computes content fingerprints. ReviewStamp compares the reviewed fingerprint with the current one. A fingerprint is a hash identifying the exact canonical bytes; it does not establish what those bytes mean. [CL6] [CL10]

C4 · Code within quality gates

Scope: Quality gates · Gates



C4: these four classes exist in gates.py. GateEngine returns Finding records and uses ArtifactStore and ReviewStamp to check content and review freshness. This view deliberately selects one risk-bearing component.

This code view is deliberately narrow. It explains a consequential mechanism rather than pretending to document every line of the repository. Proposed future code would need to be labeled proposed. The renderer and host adapters have their own responsibilities and are not hidden inside this component.

The loop, its contracts and its failure behavior

I organize execution into eight stages: frame, research, challenge, design, story, draft, review and export. In each stage the host reads current artifacts, decides the necessary work, uses its permitted tools, inspects results and updates the records. The coordinator then checks the cumulative prerequisites. Successful checks allow the next stage; a substantive correction reopens the earliest affected stage.

The first contract is between the host and coordinator. Commands receive a run path. Read-only commands report status, findings or a fingerprint. A successful advance mutates the stage. A failed gate returns exit code 1; invalid operations or exhausted budgets return code 2. If the host loses a response, it should inspect the current state before repeating a mutation. Repeating an advance is not an idempotent way to recover from uncertainty: it could advance a second stage if the first succeeded.

The second contract is the artifact format. Sources and claims belong in evidence.json; alternatives and reversal conditions in decisions.json; elements and views in architecture.json; narrative mappings in story.json. The paper uses stable section anchors. Every slide references its paper section and claims. The structure permits added fields while preserving required version-1 records. Breaking schema changes require a deliberate migration.

The third contract binds review and output. The canonical fingerprint covers the brief, framing, evidence, decisions, architecture, story, paper, slide source and generated diagram sources. Review declarations are outside that fingerprint so the act of recording a review does not invalidate itself. Exports record the same source fingerprint and a hash for each delivered file. If a reviewed PDF or deck changes, the old inspection hash no longer matches. [CL6]

In the source-correction scenario, I first reopen research with a reason. The coordinator saves a snapshot and increments the revision count. The host changes the claim, revisits the affected decision, updates the design if necessary, and revises both publications. It regenerates views after architecture changes. The previous review must fail against the new canonical fingerprint, so the host must substantively review the current content and inspect the new exports. Structural acceptance still requires semantic assessment. [CL3]

If a host stops during a stage, the next session reads the files and state rather than assuming the earlier conversation completed the work. Cumulative checks expose missing or malformed records. If a required source cannot be retrieved, a pivotal claim must remain unverified, become a qualified proposal or be removed. If rendering is unavailable, the host can provide the useful source draft, but it cannot call the required PDF and PowerPoint complete.

I bound work with configurable prompt and revision limits. The defaults, 24 issued stage prompts and three revision cycles, are operating choices rather than measured optimal values. The host must also impose time, token and cost limits because the local coordinator cannot observe hidden API consumption. After repeated failed repairs or an exhausted budget, the honest result is a blocked draft with a specific next requirement.

I retain a clear security boundary: the kit provides coordination, not independently enforced authority. A writer with unrestricted filesystem access could alter review declarations, the validator or state. Hashes help detect stale work in cooperative use; they do not make an untrusted writer trustworthy. Retrieved pages are evidence, not instructions, and credentials should never enter public artifacts. [CL8]

How I keep the paper and deck cohesive

I create the story map before prose or slide layout. Its sequence is problem, necessary conditions, choice, mechanism, limits, validation and decision. Each beat links claims, decisions, paper sections, slides and relevant views. The paper develops the argument; the deck selects the main conclusions and places implementation detail in an appendix. This is the mechanism by which I intend to reduce drift, not proof that drift has been eliminated. [CL7]

I use first-person authorial reasoning: I propose, I choose and I would validate. I do not invent an author's measurements, experience or deployment history. Factual explanation can use normal declarative sentences. Every acronym and unfamiliar concept must be explained before the reader needs it; neither artifact relies on the reader having seen the conversation or this kit's working notes.

Every slide has a conclusion, a linked paper section, claim references and speaker notes. Simplification must preserve decision-changing conditions. For example, "the kit is portable" becomes "the workflow is portable across hosts with the necessary capabilities," with untested integration clearly stated. Targets remain targets, proposals remain proposals, and future experiments remain future experiments.

All four architecture views appear in both publications. C1 and C2 carry the main deck narrative; C3 and C4 sit in its implementation appendix. A newcomer should be able to trace the same source-correction example from system boundary to local review code without encountering a renamed or unexplained component.

How I would implement, operate and validate it

I would begin with one representative brief and a human-checklist baseline. The author would agree on material-defect definitions, acceptable effort and reader-comprehension questions before generating the documents. This prevents the acceptance criteria from being weakened after an attractive output appears. [CL11]

The first increment is a complete set of explicit context, evidence and decisions. The second adds shared views, the story map and both publication forms. The third tests whether the added structure improves the work. Only after that would I add a managed service, stronger approvals or collaboration, and only in response to a demonstrated need.

I separate validation into three layers. Local regression tests check mechanisms such as blocked unsupported facts, broken references, invalid diagram nesting, stale review records and exhausted budgets. The release's VALIDATION.md records the exact tests and results. Host acceptance runs must establish that the selected Claude Code and Copilot configurations can retrieve sources, resume a run and export the outputs. A comparative reader evaluation must assess whether the publication and decision quality improve. Success in the first layer cannot establish the other two.

For that comparison I would use representative briefs, record active author and reviewer time, count material defects, and ask readers to explain the mechanism, tradeoff and reversal trigger. A separate reader who has not seen which process produced the work would be preferable. With a small pilot I would report observations and uncertainty, not claim a universal effect size.

Operating ownership stays simple. One author owns a run; a maintainer owns protocol and schema changes. Status history, command findings, review declarations and hashes provide an audit trail for cooperative work. Source retention and snapshots follow the author's policy. Snapshots copy the run, excluding earlier snapshots, so storage can grow with output size and revision count. Hashing cost grows with canonical byte size. No throughput, cost saving or production service level is claimed.

I would evolve the system through stable record contracts and a regression set of actual failures. A new model, tool or prompt version should be compared against those failures and fresh examples. General lessons may inform the next protocol version, but model-written memory should not become a verified source merely because it persists.

What would change my recommendation

I would choose the checklist if the structured loop added more work without improving material defects or reader understanding. I would choose a managed service if concurrent authors, enforceable reviewer identity or independent approval became necessary. I would narrow a portability claim if a required host capability could not be demonstrated. These are the reversal conditions that keep the design open to disconfirmation. [CL8] [CL11]

My immediate decision request is to select one recurring architecture problem, name the baseline and agree on the pilot's acceptance criteria. The result of this release is a working local kit and a worked example, ready for review. It is not evidence that an agent can replace accountable judgment or that a longer workflow always produces better architecture.

Terms and references

- Canonical record: the authoritative record from which publication artifacts are derived.
- C4: the software structure model with context, container, component and code views.
- Gate: a check that blocks a transition when a requirement is unmet.
- Fingerprint: a hash of the exact current canonical bytes.
- Semantic review: assessment of meaning, evidence and decision quality.
- Disconfirmation: an attempt to find evidence that defeats or changes a claim or decision.
- Idempotency: whether repeating an operation changes its effect.

Claim markers such as [CL1] refer to evidence.json. Source markers below identify the retrieved primary references. All were accessed on 6 October 2026. The specific workflow, thresholds and scripts are this kit's design, not prescriptions from those sources.

[S1] C4 model. [Diagrams](#).

[S2] C4 model. [System context diagram](#).

[S3] C4 model. [Container diagram](#).

[S4] C4 model. [Component diagram](#).

[S5] C4 model. [Code diagram](#).

[S6] C4 model. [Notation](#).

[S7] Anthropic. [Building effective agents](#). 19 December 2024.

[S8] Anthropic. [Demystifying evals for AI agents](#). 9 January 2026.

[S9] Anthropic. [How Claude remembers your project](#).

[S10] GitHub. [Adding repository custom instructions in your IDE](#).

[S11] Architecture Loop 1.0. Supplied source: loop.py, gates.py, diagrams.py and render.py. Prototype implementation, inspected as part of this release. See VALIDATION.md for the actual test and rendering checks.